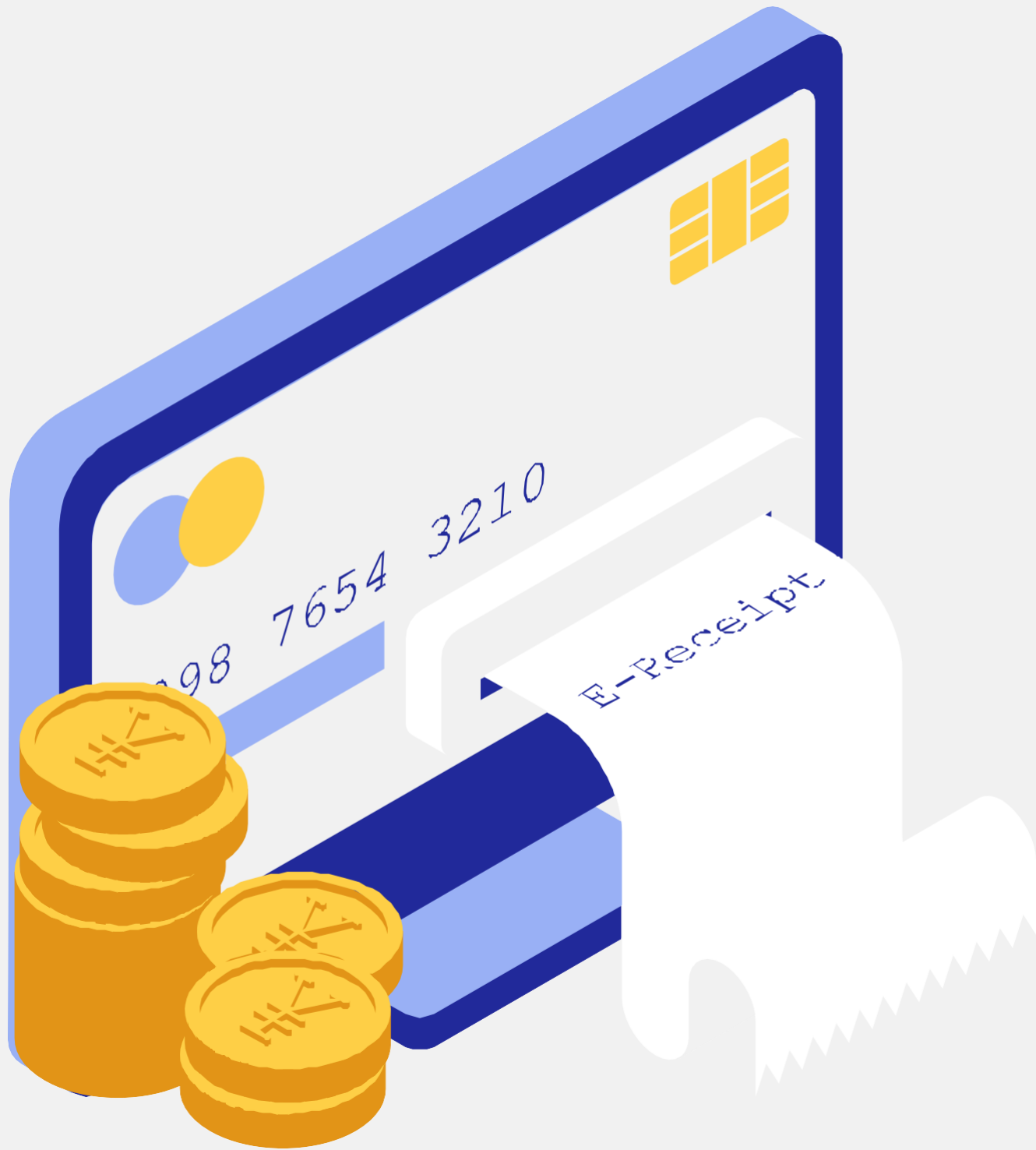


ISSS625 Query Processing and Optimisation

Window Function Query Optimization

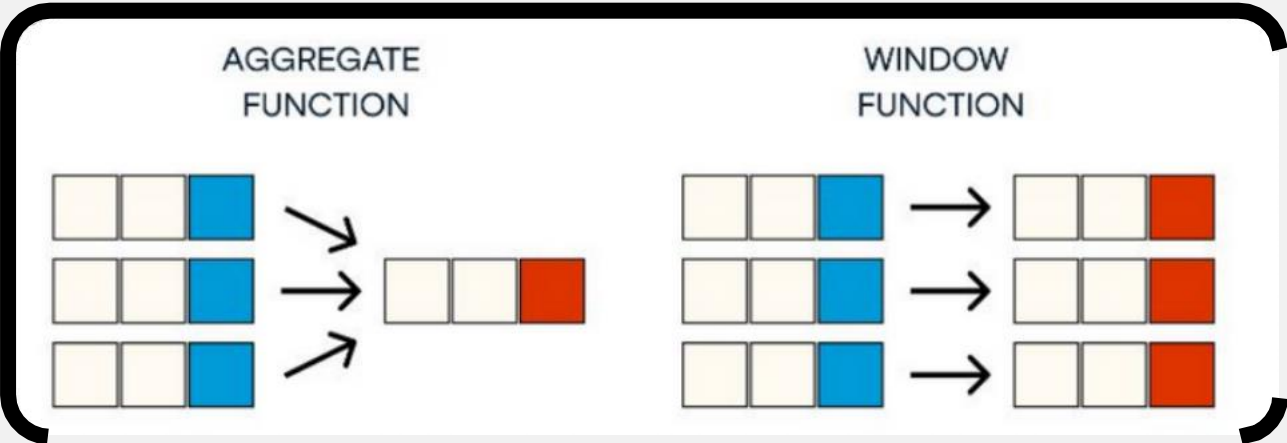


WINDOW FUNCTION

WINDOW FUNCTION() OVER (PARTITION BY group_expression ORDER BY sort_expression)

A window function performs a calculation across a set of table rows that are related to the current row. Below are the main components of a window function

- **OVER:** This clause defines the window over which the function operates
- **PARTITION BY:** Divides the result set into partitions to which the function is applied. If omitted, the function treats the entire result set as a single partition
- **ORDER BY:** Specifies the order of rows in each partition



Aggregate functions	compress data → one row per group
Window functions	retains each row data → adds calculated columns to each row

Types of Window Function :

- **Aggregate Window Function:** Calculate aggregates over a window of rows while retaining individual rows
Example : SUM(), AVG(), COUNT(), MAX(), MIN()
- **Ranking Window Function:** These functions provide rankings of rows within a partition based on specific criteria
Example : RANK() , DENSE_RANK(), ROW_NUMBER(), PERCENT_RANK()
- **Value Window Function:** Functions that retrieve a specific value from a single row within a defined set of related rows
Example : LAG(), LEAD(), FIRST_VALUE(), LAST_VALUE(), NTH_VALUE

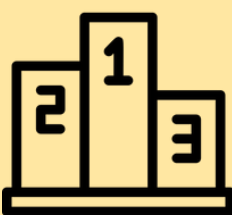
WINDOW FUNCTION

Ranking Functions



RANK()

Assigns ranks to rows giving tied values the same rank but skipping subsequent ranks



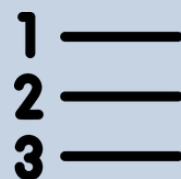
DENSE_RANK()

Assigns rank with no gaps and the tied values share the same rank but ranks remain consecutive.



PERCENT_RANK()

Calculates the relative rank of each row as a percentage within its partition



ROW_NUMBER()

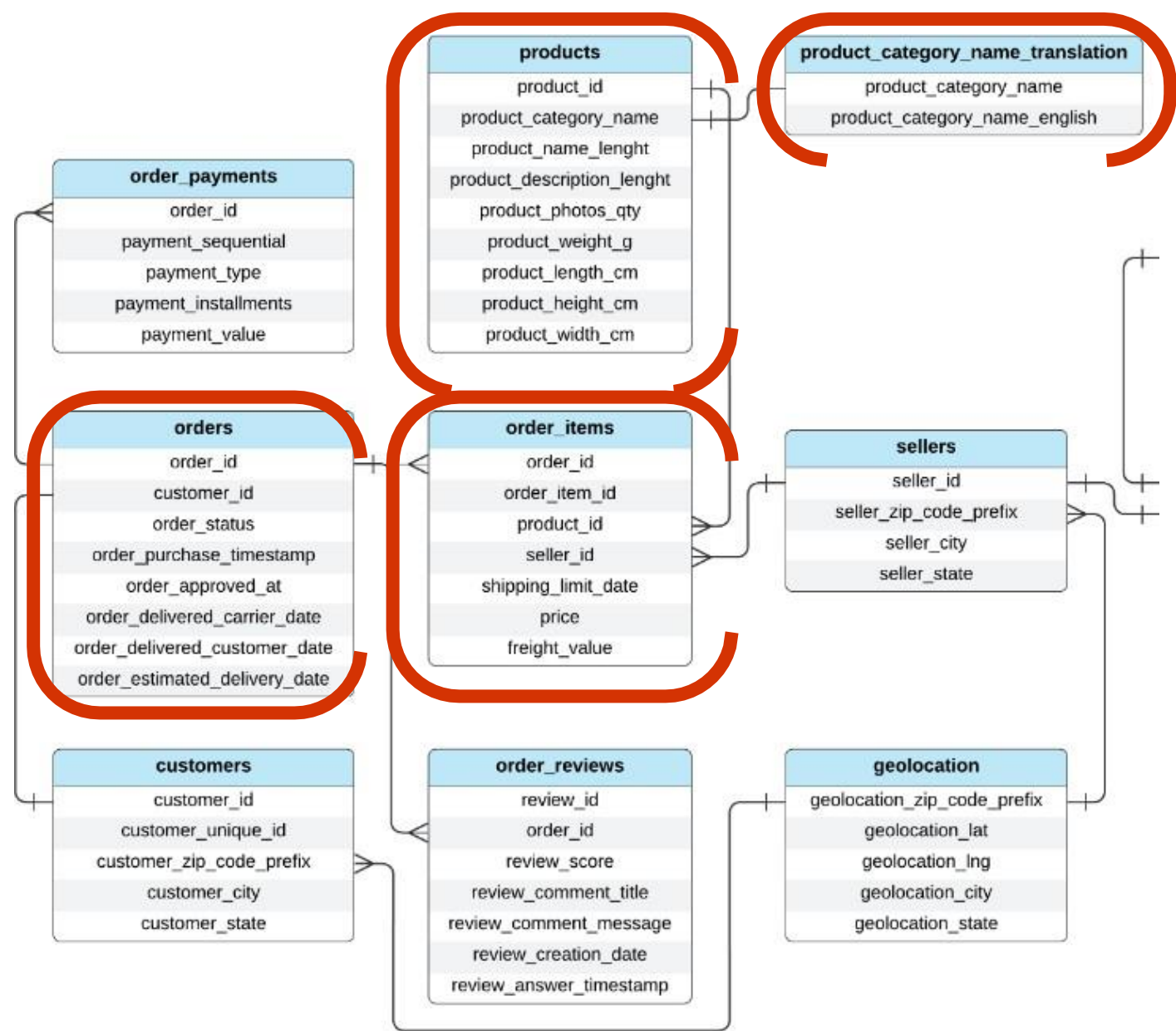
Assigns a unique sequential number to each row based on the column ordering specified

	name	salary	dense_rank	rank	row_number	percent_rank
▶	Clara	150000	1	1	1	0
	Noah	150000	1	1	2	0
	Liam	120000	2	3	3	0.5
	Mia	90000	3	4	4	0.75
	Ethan	50000	4	5	5	1

Dataset Overview

Olist E-commerce Dataset (.sqlite)

<https://www.kaggle.com/datasets/terencicp/e-commerce-dataset-by-olist-as-an-sqlite-database/data>



Business Goal

Identify the **top selling product** in **each category** from **all states**.

product_category_name	product_category_name_english	product_id	units_sold
moveis_decoracao	furniture_decor	aca2eb7d00ea1a7b8ebd4e68314663af	527
cama_mesa_banho	bed_bath_table	99a4788cb24856965c36a24e339b6058	488
ferramentas_jardim	garden_tools	422879e10f46682990de24d770e7f83d	484
informatica_acessorios	computers_accessories	d1c427060a0f73f6b889a5c7c61f2ac4	343
relogios_presentes	watches_gifts	53b36df67ebb7c41585e8d54d6772e08	323
beleza_saude	health_beauty	154e7e31ebfa092203795c972e5804a6	281
utilidades_domesticas	housewares	42a2c92a0979a949ca4ea89ec5c7b934	183
casa_conforto	home_comfort	35afc973633aaeb6b877ff57b2793310	165
consoles_games	consoles_games	0aabfb375647d9738ad0f7b4ea3653b1	142

Importing Dataset

- **Converted the SQLite dataset to MySQL compatible format**
 - TEXT >> VARCHAR(255)
 - INTEGER >> INT, REAL >> FLOAT
 - AUTOINCREMENT >> AUTO_INCREMENT
- **Imported the dataset into MySQL:**



Unoptimized Query:

```
WITH counts AS (
  SELECT
    p.product_category_name,
    i.product_id,
    COUNT(*) AS units_sold
  FROM orders o
  JOIN order_items i USING (order_id)
  JOIN products p ON p.product_id = i.product_id
  WHERE p.product_category_name IS NOT NULL
  GROUP BY p.product_category_name, i.product_id
)
SELECT
  c1.product_category_name,
  t.product_category_name_english,
  c1.product_id,
  c1.units_sold
FROM counts c1
LEFT JOIN counts c2
  ON c2.product_category_name = c1.product_category_name
  AND (
    c2.units_sold > c1.units_sold
    OR (c2.units_sold = c1.units_sold AND c2.product_id < c1.product_id)
  )
LEFT JOIN product_category_name_translation t
  ON t.product_category_name = c1.product_category_name
WHERE c2.product_id IS NULL
ORDER BY c1.units_sold DESC, c1.product_category_name, c1.product_id;
```

1

Count the number of units sold for each product.

2

Find the product with the highest number of units sold for each product category.

Optimized Query:

```
SELECT
  r.product_category_name,
  t.product_category_name_english,
  r.product_id,
  r.units_sold
FROM (
  SELECT
    p.product_category_name,
    i.product_id,
    COUNT(*) AS units_sold,
    DENSE_RANK() OVER (
      PARTITION BY p.product_category_name
      ORDER BY COUNT(*) DESC, i.product_id
    ) AS rnk
  FROM orders o
  JOIN order_items i USING (order_id)
  JOIN products p ON p.product_id = i.product_id
  WHERE p.product_category_name IS NOT NULL
  GROUP BY p.product_category_name, i.product_id
) AS r
LEFT JOIN product_category_name_translation t
  ON t.product_category_name = r.product_category_name
WHERE r.rnk = 1
ORDER BY r.units_sold DESC, r.product_category_name, r.product_id;
```

1. Perform a self-join; comparison between c1 and c2 (c2 units_sold > c1 units_sold) in each product category, the current row is being compared with other rows on units sold.
2. If there is no better competitor, c2 fields in the joined table will be NULL.
3. Filtering for NULL in c2 -> identify the product_id on c1 which has the highest number of units sold (aka anti-join).

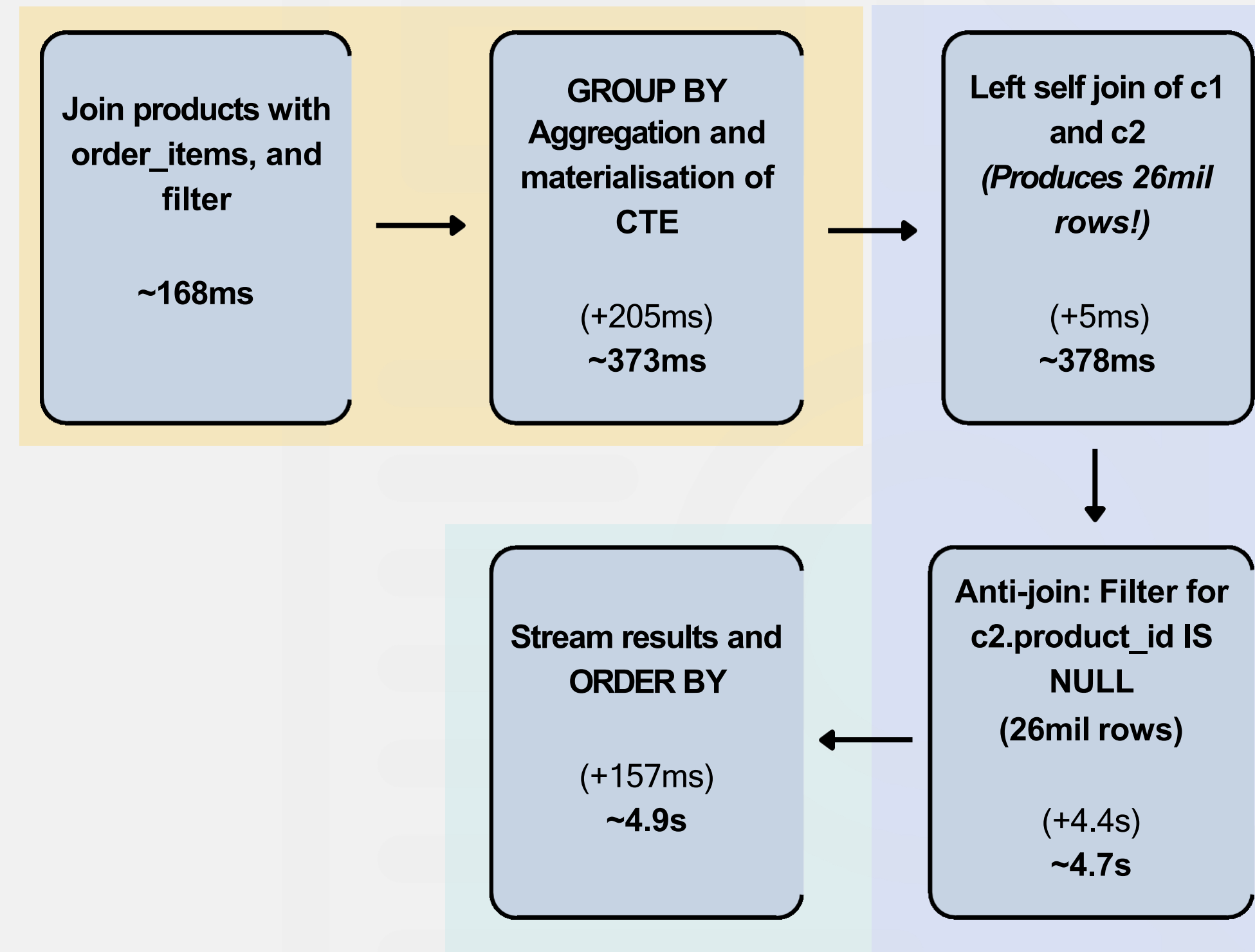
Complexity: Each product in a category compares against every other product (n^2 comparisons), quadratic - $O(n^2)$

1. PARTITION BY groups rows according to product category.
2. Window ranking function (sorts) and ranks the units sold across products in the same product category.
3. Filtering for rank = 1 identifies the product with the highest number of units sold.

Complexity: Sorting once, linearithmic - $O(n \log n)$

Unoptimized Query using EXPLAIN ANALYZE

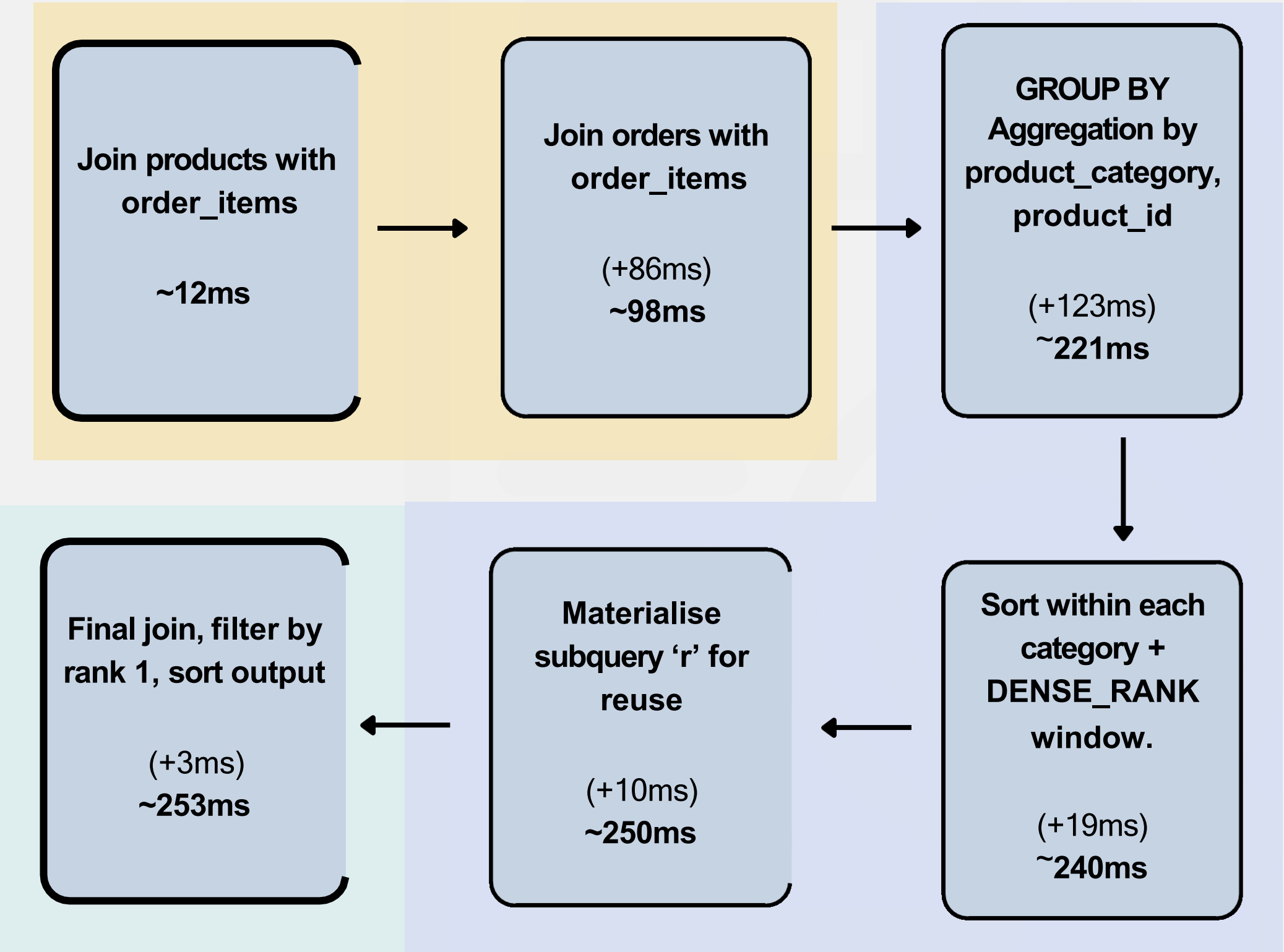
```
WITH counts AS (  
  SELECT  
    p.product_category_name,  
    i.product_id,  
    COUNT(*) AS units_sold  
  FROM orders o  
  JOIN order_items i USING (order_id)  
  JOIN products p ON p.product_id = i.product_id  
  WHERE p.product_category_name IS NOT NULL  
  GROUP BY p.product_category_name, i.product_id  
)  
SELECT  
  c1.product_category_name,  
  t.product_category_name_english,  
  c1.product_id,  
  c1.units_sold  
FROM counts c1  
LEFT JOIN counts c2  
  ON c2.product_category_name = c1.product_category_name  
  AND (  
    c2.units_sold > c1.units_sold  
    OR (c2.units_sold = c1.units_sold AND c2.product_id < c1.product_id)  
  )  
LEFT JOIN product_category_name_translation t  
  ON t.product_category_name = c1.product_category_name  
WHERE c2.product_id IS NULL  
ORDER BY c1.units_sold DESC, c1.product_category_name, c1.product_id;
```



Note: May have overlapping operations

Optimized Query using EXPLAIN ANALYZE

```
SELECT
  r.product_category_name,
  t.product_category_name_english,
  r.product_id,
  r.units_sold
FROM (
  SELECT
    p.product_category_name,
    i.product_id,
    COUNT(*) AS units_sold,
    DENSE_RANK() OVER (
      PARTITION BY p.product_category_name
      ORDER BY COUNT(*) DESC, i.product_id
    ) AS rnk
  FROM orders o
  JOIN order_items i USING (order_id)
  JOIN products p ON p.product_id = i.product_id
  WHERE p.product_category_name IS NOT NULL
  GROUP BY p.product_category_name, i.product_id
) AS r
LEFT JOIN product_category_name_translation t
  ON t.product_category_name = r.product_category_name
WHERE r.rnk = 1
ORDER BY r.units_sold DESC, r.product_category_name, r.product_id;
```



Note: May have overlapping operations

Comparison between Optimized and Unoptimized (95% reduction in runtime)

	Window Functions (DENSE_RANK)	Efficiency

* Of the additional memory used, 82% was for higher main thread memory (THD::main_mem_root), 9% was for synchronisation (rwlock_instance), and 7% was for shared metadata structures (TABLE_SHARE::mem_root).

Benefits

- **Single-Pass, Multi-Metric Efficiency**

Reduced query time significantly
→ single partition can power several stats as once, avoiding repeated self-joins and extra sorts.

- **Clearer, Safer Logic**

More readable by simplifying query logic
→ fewer complex subqueries, easier maintenance.

- **Row-Preserving Insight**

Windows keep one result per input row (unlike GROUPBY)
→ compare row-level facts with group-level context without collapsing the dataset

Drawbacks

- **Limited Indexing Help**

Cannot fully utilise indexing because the function requires all rows of a group before ranking, whereas indexing is good at finding SOME rows.

- **Compatibility Gaps**

“Recent” support from MySQL v8.0.2 (2018).
No support in v5.7 (last stable release in 2015).

- **Expensive sorts**

Each partition must be sorted by the ORDER BY.
In event of very large datasets → Large partitions
→ big memory use and possible disk spills.

Thank You !

